

Home assessment

Wallet Ledger Service (Node.js + TypeScript)

Build a small backend service that maintains a ledger-based wallet balance per user. Host in a remote repository we can pull from.

Stack requirements:

- NodeJS + TypeScript
- ExpressJS or NestJS
- PostgreSQL + Prisma ORM (migrations required)

Overview

This service represents a small microservice that does not manage user accounts. It relies on the `x-user-id` request header to scope all operations.

A user can:

- list items (**Example items:** car, bike, tv, etc...)
- top up their balance (credits)
- purchase an item (debit)
- check their current balance

A user cannot go below 0. A purchase must fail if the user has insufficient balance.

Key considerations

- **Ledger-based balance (required)**
Balance must be calculated as the sum of all ledger transactions for the user. Do not store or maintain a `balance` column in the database.
- **Concurrency (required)**
Think about concurrent requests. For example, if two purchase requests happen at the same time, the system must not allow the balance to go below

zero.

You should implement a concurrency-safe approach. Explain your approach briefly in the README.

- **Price changes over time (required)**

Item prices may change in the future. The system must be able to track the exact price used at the moment of purchase, even if the item price changes later. This means you should persist the item price used at purchase time.

- **Database indexes (required)**

Assume this is a heavy service with a lot of ledger transactions. Add appropriate DB indexes to support the most common queries (e.g., balance calculation and purchase flows) and explain in the README:

- which indexes you added
- what query/operation each index supports
- tradeoffs (write cost vs read performance)

- **Idempotency (design note only; no implementation required)**

Imagine `POST /api/purchases` supports:

Header: `x-idempotency-key: <string>`

If the same request with the same key is sent twice, the second request should not create a second transaction and should not affect balance.

Write a short note (5–10 sentences) in the README describing how you would implement idempotency.

Notes

- Single currency: cents only (integer). All money-related operations are in cents – item price, user balance etc.
- Items are hardcoded in the codebase (no DB table needed).
- No users table:
 - Each request includes a user id header: `x-user-id: <UUID>`
 - Store `user_id` as a UUID column where needed (no FK required).
- Most likely you will have a choice picking up between 1 or 3 tables – we kindly ask you to lean towards normalization approach.

API Spec

Shared header (all endpoints)

Headers

```
{
  "x-user-id": "UUID"
}
```

If `x-user-id` is missing or not a valid UUID, return 400.

GET /api/items

Returns a list of 2–4 items. Items can be hardcoded in code.

Response body:

```
[
  {
    "id": "UUID",
    "name": "string",
    "price": 499
  }
]
```

POST /api/credits

Adds credits to the user and returns the updated balance.

Request body:

```
{
  "amount": 1000
}
```

Rules:

- `amount` must be a positive integer (`> 0`), otherwise 400
- Must create a ledger transaction for the user

- Must return the updated balance

Response body:

```
{
  "balance": 1000
}
```

POST /api/purchases

Purchases a item. Creates a debit ledger transaction equal to the item price.

Request body:

```
{
  "itemId": "UUID"
}
```

Rules:

- If item does not exist: 404
- If user has insufficient balance: 409
- Must be concurrency-safe: do not allow balance < 0 under concurrent requests
- Must persist the item price at purchase time for historical accuracy

Response:

- 204 No Content (no response body) on success

GET /api/balance

Returns the current balance for the user.

Response body:

```
{
  "balance": 501
}
```

Rules:

- If this user ID doesn't have records in the system yet, return:

```
{  
  "balance": 0  
}
```

Deliverables

- Source code
- Prisma schema + migrations
- README:
 - how to run locally
 - how to run migrations
 - concurrency approach (short explanation)
 - idempotency note (5–10 sentences)
- Double check that all requirements from "Key considerations" section is fulfilled.